RescueMesh AI

AI-Powered Emergency Response & Disaster Coordination Platform

Product Requirements Document

Nebius × NVIDIA Global AI Hackathon

Version 1.0 • Hackathon MVP

Tagline
From scattered emergency signals to coordinated rescue missions.

1. Product Overview

1.1 Product Name

RescueMesh AI

1.2 Tagline

From scattered emergency signals to coordinated rescue missions.

1.3 One-Line Description

RescueMesh AI is a multi-agent emergency response platform that collects fragmented disaster information, verifies and prioritizes incidents, identifies required resources, recommends routes and medical facilities, and generates an actionable rescue mission plan for a human emergency operator.

2. Problem Statement

During disasters and large-scale emergencies, information arrives from multiple sources: emergency reports, text messages, images/videos, social media, weather information, road/traffic information, hospital availability, rescue-team reports, and drone/sensor information.

The information is often unstructured, incomplete, duplicated, conflicting, time-sensitive, and geographically distributed.

Emergency operators therefore need to manually process large amounts of information before deciding: What happened? Where? How serious is it? Who needs help first? What resources are required? Which route should responders take? Where should victims be transported?

RescueMesh AI aims to reduce this information-processing burden by using a coordinated multi-agent AI system.

3. Product Vision

Build an AI-powered emergency operations center where multiple AI agents continuously transform raw emergency information into verified, prioritized and explainable rescue missions.

The human operator remains responsible for approving missions.

4. Goals

G1 — Automatic Incident Detection

Convert unstructured reports into structured emergency incidents.

G2 — Incident Prioritization

Automatically determine which incidents require immediate attention using number of people affected, medical emergency, immediate danger, environmental conditions, available evacuation routes, and resource requirements.

G3 — Multi-Agent Investigation

Use specialized Triage, Route, Medical, Resource, and Verification agents.

G4 — Evidence-Based Recommendations

Important AI recommendations should contain supporting evidence wherever possible.

G5 — Mission Generation

Convert an analyzed incident into an actionable rescue mission.

G6 — Human-in-the-Loop Safety

Every mission should require human approval.

5. Non-Goals

The MVP will not automatically contact real emergency services, dispatch real rescue teams, provide medical diagnosis, replace emergency command authorities, guarantee route safety, guarantee hospital availability, or make autonomous life-or-death decisions.

The project is a decision-support and coordination prototype.

6. Target Users

Primary — Emergency Operations Center Operator

Monitor incidents, review AI analysis, approve missions, assign resources, and track active rescue operations.

Secondary — Rescue Team

Receives mission details, location, priority, required equipment, and recommended route.

Secondary — Medical Coordinator

Receives injured-person information, critical requirements, recommended hospitals, and estimated transport requirements.

7. Core User Journey

Emergency report arrives → AI extracts information → Incident created → Duplicate reports grouped → Incident severity calculated → Specialized agents investigate → Evidence collected → Rescue plan generated → Human operator reviews → Operator approves → Mission becomes ACTIVE → Mission progress tracked.

8. Core Features

Feature 1 — Live Incident Feed

Display incoming emergency reports and automatically correlate related reports.

Feature 2 — AI Incident Extraction

Convert text into structured JSON containing incident type, people affected, medical emergency, location, resources, and confidence.

Feature 3 — Incident Clustering

Group multiple reports describing the same event to prevent duplicate work.

Feature 4 — Triage Agent

Determine incident priority, reasons, risk factors, and confidence.

Feature 5 — Route Agent

Determine possible responder routes using incident location, road status, blocked roads, and rescue-team location.

Feature 6 — Medical Agent

Evaluate emergency medical requirements and suitable hospitals from the demo database.

Feature 7 — Resource Agent

Determine required rescue and medical resources.

Feature 8 — Verification Agent

Check important claims and classify them as VERIFIED, PARTIALLY VERIFIED, UNVERIFIED, or CONFLICTING.

Feature 9 — Evidence Panel

Show supporting evidence for important AI recommendations.

Feature 10 — Mission Planner

Generate a mission containing priority, location, people, resources, route, destination, confidence, and approval controls.

Feature 11 — Human Approval

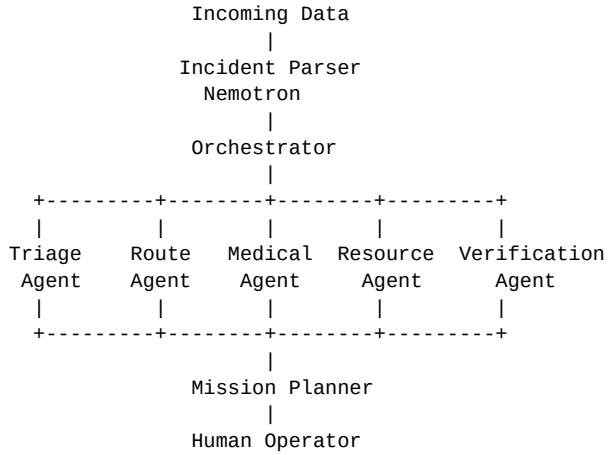
Operator can APPROVE, MODIFY, or REJECT an AI-generated plan.

Feature 12 — Live Mission Tracking

Track mission states from PENDING through COMPLETED.

9. AI Architecture

The system uses a multi-agent architecture rather than asking a single LLM to decide everything.



10. NVIDIA / Nebius Integration

NVIDIA Nemotron

Primary reasoning and language intelligence layer for incident extraction, classification, agent reasoning, planning, evidence synthesis, and mission generation.

Nebius

Infrastructure for AI workloads and/or inference services.

Tavily

Optional live web research for evidence gathering and verification.

11. Recommended Technology Stack

Frontend

React, TypeScript, Tailwind CSS, map visualization, Recharts.

Backend

Python, FastAPI, Pydantic, AsyncIO.

AI

NVIDIA Nemotron, Nebius Token Factory, agent orchestration, RAG, structured outputs.

Database

PostgreSQL; optional pgvector for vector search/RAG.

External Services

Tavily, weather API, map/route API. Simulated data can be used for the MVP.

12. Database Design

Users

id, name, email, role, created_at

Incidents

id, type, description, latitude, longitude, people_affected, severity, medical_emergency, status, confidence, created_at, updated_at

Reports

id, incident_id, source, content, location, timestamp, confidence

Resources

id, type, quantity, location, status

Missions

id, incident_id, priority, route, destination, status, confidence, approved_by, created_at

Evidence

id, incident_id, source, claim, content, confidence, timestamp

13. API Design

POST /api/incidents

Create incident.

GET /api/incidents

Get incidents.

POST /api/incidents/{id}/analyze

Analyze incident.

POST /api/incidents/{id}/agents

Run agents.

POST /api/incidents/{id}/mission

Generate mission.

POST /api/missions/{id}/approve

Approve mission.

POST /api/missions/{id}/reject

Reject mission.

GET /api/missions/{id}

Get mission status.

14. Frontend Pages

Page 1 — Command Center

Main dashboard containing incidents, map, and AI analysis.

Page 2 — Incident Details

Incident information, reports, AI analysis, severity, evidence, agent activity, and recommended mission.

Page 3 — Mission Control

Mission, resources, route, destination, team, status, confidence, evidence, and approval controls.

Page 4 — Agent Activity

Visible multi-agent processing timeline for the hackathon demo.

15. Demo Scenario

A flood emergency occurs. Reports state that water has entered houses in Sector 12, approximately 30–40 people are trapped, one elderly person needs oxygen, and the main road is blocked.

Nemotron extracts structured information; reports are clustered; Triage marks the incident CRITICAL; Medical detects oxygen requirements; Route identifies a recommended route; Resource determines boat, oxygen, ambulance and responders; Verification checks evidence; Mission Planner generates the rescue plan; the operator approves it.

16. MVP Scope

Must Have

Dashboard; incident creation; AI incident extraction; prioritization; multi-agent analysis; resource recommendation; route recommendation; evidence display; mission generation; human approval; Nebius integration; NVIDIA

Nemotron integration.

Should Have

Tavily integration; interactive map; simulated live reports; mission tracking; PostgreSQL persistence.

Nice to Have

Image analysis; drone footage; voice reports; real-time streaming; multiple simultaneous disasters.

17. Success Metrics

AI Accuracy

Percentage of incident fields correctly extracted.

Incident Deduplication

Number of duplicate reports successfully grouped.

Planning Latency

Target under 30 seconds for the demo pipeline.

Evidence Coverage

Percentage of important recommendations containing supporting evidence.

Human Approval

Percentage of AI-generated missions requiring manual modification; this is a demo metric, not a real-world operational benchmark.

18. Safety Requirements

SR1

AI must clearly label recommendations as AI-generated.

SR2

Human approval is required before mission activation.

SR3

Unverified information must be explicitly labeled.

SR4

Conflicting information must not be silently resolved.

SR5

The system must not claim certainty where evidence is unavailable.

19. Security Requirements

Authenticate users; restrict operator functionality; protect incident data; avoid exposing sensitive personal information; log mission approvals/rejections; validate API inputs.

20. Performance Requirements

Dashboard load

< 2 seconds

Incident creation

< 1 second

AI analysis

< 15–30 seconds

Mission generation

< 10 seconds

Actual AI latency will depend on model and infrastructure configuration.

21. Failure Handling

If an AI agent fails, retry; use a fallback where possible; mark unavailable; and continue with remaining agents.

Example: if the Route Agent fails, the mission may still be generated but the route recommendation is marked unavailable and manual routing is required.

22. Observability

Backend should log request, agent, model, latency, token usage, result, error, and confidence. This helps demonstrate genuine use of the AI infrastructure.

23. Agent Prompt Architecture

Each agent should have a dedicated role. Prompts should instruct agents not to invent facts, use only provided information, and return structured JSON with relevant fields such as severity, reasons, risk factors, confidence, and missing information.

24. Important Design Principle

Do not simply ask one LLM: “What should we do?” Instead use specialized agents, independent analysis, evidence, cross-checking, and a mission planner.

25. Competitive Differentiation

1. Multi-Agent Architecture

Not a single chatbot.

2. Evidence-Based AI

Recommendations include supporting evidence.

3. Human-in-the-Loop

AI recommends; humans approve.

4. Mission Generation

Final output is an operational plan rather than just text.

5. Real-Time Simulation

Incoming reports can continuously update the situation.

6. Explainability

Operators can inspect why the system marked an incident critical.

26. Future Roadmap

Phase 2

Image understanding, drone imagery, voice reports, real-time weather, real-time road conditions.

Phase 3

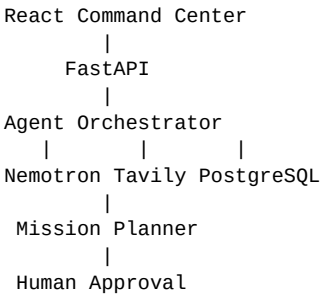
Predictive disaster modeling, resource optimization, rescue-team positioning, hospital capacity prediction.

Phase 4

Robotics integration, UAV integration, edge AI, autonomous sensor networks.

27. Hackathon Demo Architecture

React UI → FastAPI Backend → Agent Orchestrator → Nebius / NVIDIA Nemotron + Tavily + PostgreSQL → Mission Planner → Human Approval.



28. Final Product Statement

RescueMesh AI is an AI-powered emergency coordination platform that transforms fragmented disaster reports into verified, prioritized and explainable rescue missions using a multi-agent architecture powered by NVIDIA Nemotron and deployed through Nebius infrastructure.

Core flow: CHAOS → UNDERSTANDING → VERIFICATION → PRIORITIZATION → COORDINATION → HUMAN DECISION → ACTION.